```
import pandas as pd
```

```
from google.colab import files

uploaded = files.upload()
```

Choose files usa_housing_kaggle.csv
usa_housing_kaggle.csv(text/csv) - 13190 bytes, last modified: 21/01/2026 - 100% done
Saving usa_housing_kaggle.csv to usa_housing_kaggle.csv

Double-click (or enter) to edit

Read Data from Different Formats (CSV)

```
df = pd.read_csv(list(uploaded.keys())[0])
```

```
df.head()
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	SchoolRating
0	221958	1	1.9	4827	1979	2	1.45	82240	48.60	5
1	771155	2	2.0	1035	1987	2	1.75	74315	92.03	9
2	231932	1	3.0	2769	1982	1	1.46	79249	52.08	3
3	465838	3	3.3	2708	1907	3	1.62	80587	61.65	1
4	359178	4	3.4	1175	1994	2	0.74	20756	15.66	4

Next steps: Generate code with df New interactive sheet

Check Shape of the Dataset

```
df.shape
```

```
(300, 10)
```

Find missing values

```
df.isnull().sum()
```

	0
Price	0
Bedrooms	0
Bathrooms	0
SquareFeet	0
YearBuilt	0
GarageSpaces	0
LotSize	0
ZipCode	0
CrimeRate	0
SchoolRating	0

dtype: int64

Display Column Names

```
df.columns
```

```
Index(['Price', 'Bedrooms', 'Bathrooms', 'SquareFeet', 'YearBuilt',
       'GarageSpaces', 'LotSize', 'ZipCode', 'CrimeRate', 'SchoolRating'],
      dtype='object')
```

Check Data Types of Each Column

```
df.dtypes
```

	0
Price	int64
Bedrooms	int64
Bathrooms	float64
SquareFeet	int64
YearBuilt	int64
GarageSpaces	int64
LotSize	float64
ZipCode	int64
CrimeRate	float64
SchoolRating	int64

dtype: object

Find Missing Values

```
df.isnull().sum()
```

	0
Price	0
Bedrooms	0
Bathrooms	0
SquareFeet	0
YearBuilt	0
GarageSpaces	0
LotSize	0
ZipCode	0
CrimeRate	0
SchoolRating	0

dtype: int64

Basic Statistical Description

Finding out zero values

```
(df == 0).sum()
```

	0
Price	0
Bedrooms	0

For only numeric columns:

```
df.select_dtypes(include='number').eq(0).sum()
```

	0
YearBuilt	0
Price	0
LotSize	0
Bedrooms	0
Bathrooms	0
CrimeRate	0
SquareFeet	0
YearBuilt	0
GarageSpaces	72
LotSize	0
ZipCode	0
CrimeRate	0
SchoolRating	0

dtype: int64

```
df.describe()
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	Scho
count	300.000000	300.000000	300.000000	300.00000	300.000000	300.000000	300.000000	300.000000	300.000000	30
mean	522761.916667	2.966667	2.464000	3052.75000	1962.616667	1.513333	1.062200	54324.770000	49.324033	
std	256922.348275	1.451175	0.886184	1194.97144	36.005431	1.116832	0.543896	24718.938552	28.833220	
min	112666.000000	1.000000	1.000000	840.00000	1902.000000	0.000000	0.110000	10163.000000	0.190000	
25%	299936.500000	2.000000	1.700000	2053.50000	1931.750000	1.000000	0.570000	34175.000000	22.547500	
50%	505806.500000	3.000000	2.400000	3127.00000	1961.000000	1.000000	1.085000	54896.500000	49.970000	
75%	744603.750000	4.000000	3.200000	4081.00000	1994.000000	3.000000	1.482500	74108.000000	74.402500	
max	999684.000000	5.000000	4.000000	4997.00000	2022.000000	3.000000	2.000000	99881.000000	99.710000	

Indexing and Selecting Data

```
df.head()
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	SchoolRating
0	221958	1	1.9	4827	1979	2	1.45	82240	48.60	5
1	771155	2	2.0	1035	1987	2	1.75	74315	92.03	9
2	231932	1	3.0	2769	1982	1	1.46	79249	52.08	3
3	465838	3	3.3	2708	1907	3	1.62	80587	61.65	1
4	359178	4	3.4	1175	1994	2	0.74	20756	15.66	4

Next steps: Generate code with df New interactive sheet

```
df.tail()
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	SchoolRating
295	862002	4	2.8	1129	1944	0	1.80	35227	32.30	3
296	242483	3	2.4	4440	1988	2	0.46	15978	99.71	9
297	758271	2	2.2	4694	1952	2	0.71	97527	97.98	7
298	314835	5	2.0	4907	1941	3	0.53	27153	65.61	8
299	501896	5	3.8	2141	1999	0	0.77	69765	12.98	9

Specific rows and columns

```
df.iloc[0:5, 0:3]
```

	Price	Bedrooms	Bathrooms
0	221958	1	1.9
1	771155	2	2.0
2	231932	1	3.0
3	465838	3	3.3
4	359178	4	3.4

```
df.loc[0, 'Price']
```

```
np.int64(221958)
```

```
df.loc[:, ['Price', 'Bedrooms']]
```

	Price	Bedrooms
0	221958	1
1	771155	2
2	231932	1
3	465838	3
4	359178	4
...	...	...
295	862002	4
296	242483	3
297	758271	2
298	314835	5
299	501896	5

300 rows × 2 columns

Sorting Data

by one column

```
df.sort_values(by='Price')
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	SchoolRating
132	112666	2	2.6	2790	2019	0	0.69	88213	74.00	5
129	113986	2	2.1	1754	1902	2	1.37	62188	68.59	3
287	115708	4	1.4	2258	1991	3	1.98	88040	85.69	4
221	121959	1	1.7	1036	1972	0	0.59	63352	14.10	1
282	122671	1	1.5	2439	1905	1	1.42	60979	73.26	9
...	...	...	...	...	...	...	...	...	...	...
74	996942	4	2.4	4125	1938	1	0.78	71929	74.95	7
162	997421	5	2.5	3857	1945	3	1.96	76918	39.88	4
252	998613	5	1.6	4429	1904	1	1.26	46224	28.24	2
12	999159	1	2.1	4650	1971	2	1.10	55113	16.37	6
155	999684	1	3.4	3132	1960	1	1.35	16406	90.39	8

300 rows × 10 columns

descending order

```
df.sort_values(by='Price', ascending=False)
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	SchoolRating
155	999684	1	3.4	3132	1960	1	1.35	16406	90.39	8
12	999159	1	2.1	4650	1971	2	1.10	55113	16.37	6
252	998613	5	1.6	4429	1904	1	1.26	46224	28.24	2
162	997421	5	2.5	3857	1945	3	1.96	76918	39.88	4
74	996942	4	2.4	4125	1938	1	0.78	71929	74.95	7
...	...	...	...	...	...	...	...	...	...	...
282	122671	1	1.5	2439	1905	1	1.42	60979	73.26	9
221	121959	1	1.7	1036	1972	0	0.59	63352	14.10	1
287	115708	4	1.4	2258	1991	3	1.98	88040	85.69	4
129	113986	2	2.1	1754	1902	2	1.37	62188	68.59	3
132	112666	2	2.6	2790	2019	0	0.69	88213	74.00	5

300 rows × 10 columns

Sort by multiple columns

```
df.sort_values(by=['Price', 'Bathrooms'], ascending=[False, True])
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	SchoolRating
155	999684	1	3.4	3132	1960	1	1.35	16406	90.39	8
12	999159	1	2.1	4650	1971	2	1.10	55113	16.37	6
252	998613	5	1.6	4429	1904	1	1.26	46224	28.24	2
162	997421	5	2.5	3857	1945	3	1.96	76918	39.88	4
74	996942	4	2.4	4125	1938	1	0.78	71929	74.95	7
...	...	...	...	...	...	...	...	...	...	...
282	122671	1	1.5	2439	1905	1	1.42	60979	73.26	9
221	121959	1	1.7	1036	1972	0	0.59	63352	14.10	1
287	115708	4	1.4	2258	1991	3	1.98	88040	85.69	4
129	113986	2	2.1	1754	1902	2	1.37	62188	68.59	3
132	112666	2	2.6	2790	2019	0	0.69	88213	74.00	5

300 rows × 10 columns

Count Unique Values

```
df.nunique()
```

	0
Price	300
Bedrooms	5
Bathrooms	31
SquareFeet	285
YearBuilt	111
GarageSpaces	4
LotSize	154
ZipCode	300
CrimeRate	295
SchoolRating	9

dtype: int64

Check Format / Info of Dataset

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Price         300 non-null    int64
 1   Bedrooms      300 non-null    int64
 2   Bathrooms     300 non-null    float64
 3   SquareFeet    300 non-null    int64
 4   YearBuilt     300 non-null    int64
 5   GarageSpaces  300 non-null    int64
 6   LotSize       300 non-null    float64
 7   ZipCode       300 non-null    int64
 8   CrimeRate     300 non-null    float64
 9   SchoolRating  300 non-null    int64
dtypes: float64(3), int64(7)
memory usage: 23.6 KB
```

describe For all columns including non-numeric:

```
df.describe(include='all')
```

	Price	Bedrooms	Bathrooms	SquareFeet	YearBuilt	GarageSpaces	LotSize	ZipCode	CrimeRate	Scho
count	300.000000	300.000000	300.000000	300.00000	300.000000	300.000000	300.000000	300.000000	300.000000	30
mean	522761.916667	2.966667	2.464000	3052.75000	1962.616667	1.513333	1.062200	54324.770000	49.324033	
std	256922.348275	1.451175	0.886184	1194.97144	36.005431	1.116832	0.543896	24718.938552	28.833220	
min	112666.000000	1.000000	1.000000	840.00000	1902.000000	0.000000	0.110000	10163.000000	0.190000	
25%	299936.500000	2.000000	1.700000	2053.50000	1931.750000	1.000000	0.570000	34175.000000	22.547500	
50%	505806.500000	3.000000	2.400000	3127.00000	1961.000000	1.000000	1.085000	54896.500000	49.970000	
75%	744603.750000	4.000000	3.200000	4081.00000	1994.000000	3.000000	1.482500	74108.000000	74.402500	
max	999684.000000	5.000000	4.000000	4997.00000	2022.000000	3.000000	2.000000	99881.000000	99.710000	

Checks column names and structure.

```
df.columns
```

```
Index(['Price', 'Bedrooms', 'Bathrooms', 'SquareFeet', 'YearBuilt',
       'GarageSpaces', 'LotSize', 'ZipCode', 'CrimeRate', 'SchoolRating'],
      dtype='object')
```

Convert float to int

```
df['Bedrooms'] = df['Bedrooms'].astype(int)
```

Convert int to float

```
df['Price'] = df['Price'].astype(float)
```